
```

close all
clear all
clc

T = 1;
dt = 1e-3; % time step size
t = 0:dt:T-dt; % time vector
% fs=1/dt;
df = 1/T; % frequency step size
f = -1/(2*dt):df:1/(2*dt)-df; % frequency vector

f1 = 1;
f2 = 3;
m = sin(2*pi*f1*t)-sin(2*pi*f2*t);
factor = 20;
Ts = factor*dt; % sampling period
m_ds = downsample(m,factor);
m_s = upsample(m_ds,factor);

M = fftshift(fft(fftshift(m)))*dt;
M_s = fftshift(fft(fftshift(m_s)))*dt;

figure(1)
subplot(3,1,1)
hold on
plot(t,m,'k')
plot(t,m_s,'b')
hold off
title('m(t) and m_s(t) with T_s = 0.02')
xlabel('Time (sec)')

subplot(3,1,2)
plot(f,abs(M))
title('Spectrum of m(t)')
xlabel('Frequency (Hz)')

subplot(3,1,3)
plot(f,abs(M_s))
title('Spectrum of m_s(t) with f_s = 50 Hz')
xlabel('Frequency (Hz)')

W = 5; % Bandwidth

H_lpf = zeros(1,length(f));
H_lpf ( (f >= -W) & (f <= W) ) = 1;

figure(2)
plot(f,H_lpf)
title('Low-pass Filter in Frequency Domain')
xlabel('Frequency (Hz)')
M_recon = factor*M_s.*H_lpf;
m_recon = fftshift(ifft(fftshift(M_recon)))/dt;

```

```

figure(3)
plot(f,abs(M_recon))
title('Spectrum of Idealy Reconstructed Signal')
xlabel('Frequency (Hz)')

figure(4)
hold on
plot(t,m,'k')
plot(t,m_recon,'--r')
title('Original Signal vs. Ideally Reconstructed Signal')
legend('Original Signal','Reconstructed Signal')

% Step 7
repeat = factor;
m_s_flat = kron(m_ds, ones(1,repeat));

% Step 8
M_s_flat_ft = fftshift(fft(fftshift(m_s_flat)))*dt;

% Step 9
M_flat_recon = (M_s_flat_ft).*H_lpf;
m_flat_recon = fftshift(ifft(fftshift(M_flat_recon)))/dt;

figure(6)
hold on
plot(t,m,'k')
plot(t,m_s_flat,'b')
title('Original Signal vs. Flat-Top Sampling Signal')
xlabel('Time (sec)')
legend('Original Signal','Flat-Top Sampling Signal')
hold off

figure(7)
hold on
plot(t,m)
plot(t,m_flat_recon,'r--')
hold off
title('Original Signal vs. Flat-Top Reconstructed Signal')
xlabel('Time (sec)')
legend('Original Signal','Reconstructed Signal');

% step 10 quantization
k = [2 3 4 5];
m_max = max(m_ds);
m_min = min(m_ds);
SQNR = zeros(1,length(k));
m_quantized_flat = zeros(length(k),length(m));

for i = 1:length(k)
    L = 2^k(i);
    delta = (m_max-m_min)/L;
    level_max = m_max - delta/2;
    level_min = m_min + delta/2;

```

```

    q_levels=level_min:delta:level_max;
    m_level = round((m_ds-level_min)/delta);
    m_quantized = (m_level.*delta) + level_min;
    m_quantized(m_quantized>level_max) = level_max;
    m_quantized(m_quantized<level_min) = level_min;
    SQNR(i) = 10*log10((norm(m_ds)/norm(m_ds-m_quantized))^2);
    m_quantized_flat(i,:) = kron(m_quantized, ones(1,repeat));
end

figure(8)
subplot(2,2,1)
hold on
plot(t,m)
plot(t,m_quantized_flat(1,:))
xlabel('Time (sec)')
title('Original Signal vs. Quantized Signal L = 4')
hold off

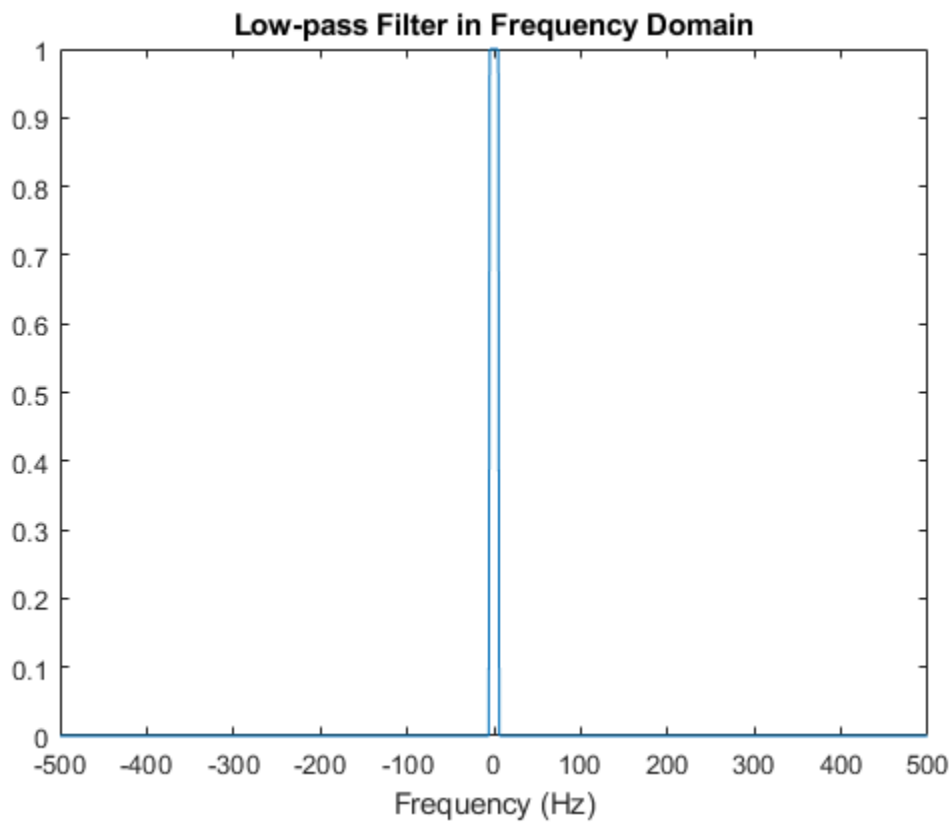
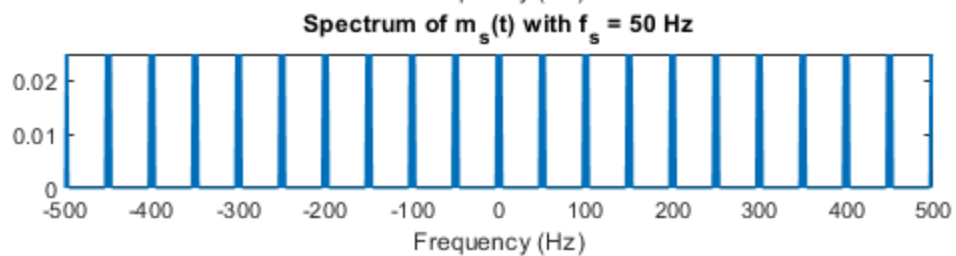
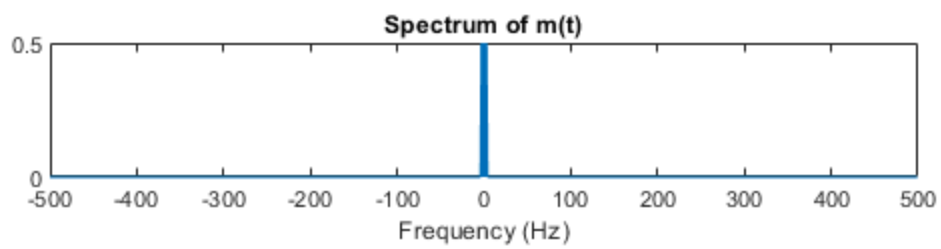
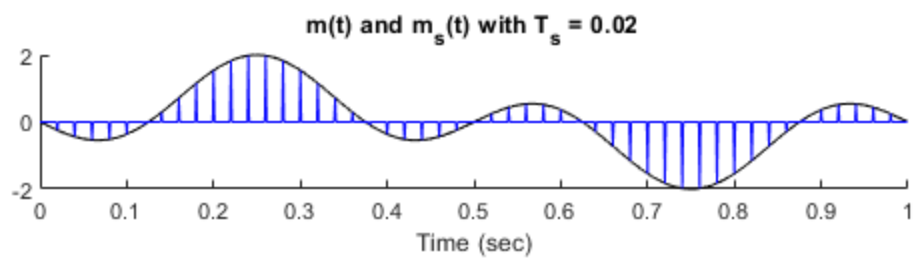
subplot(2,2,2)
hold on
plot(t,m)
plot(t,m_quantized_flat(2,:))
xlabel('Time (sec)')
title('Original Signal vs. Quantized Signal L = 8')
hold off

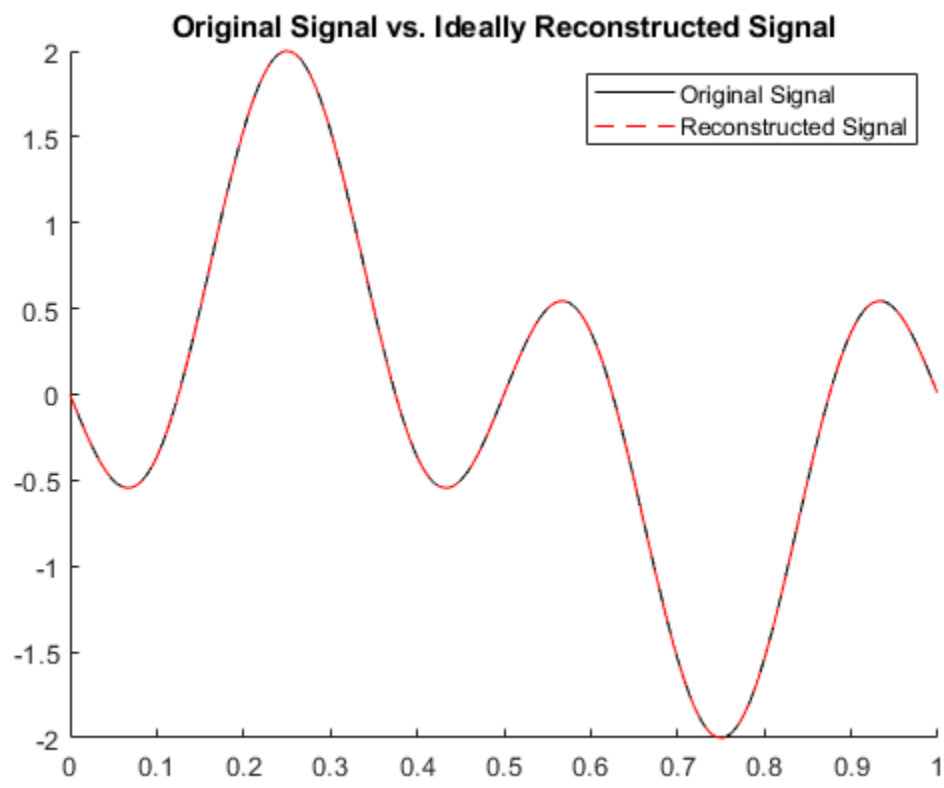
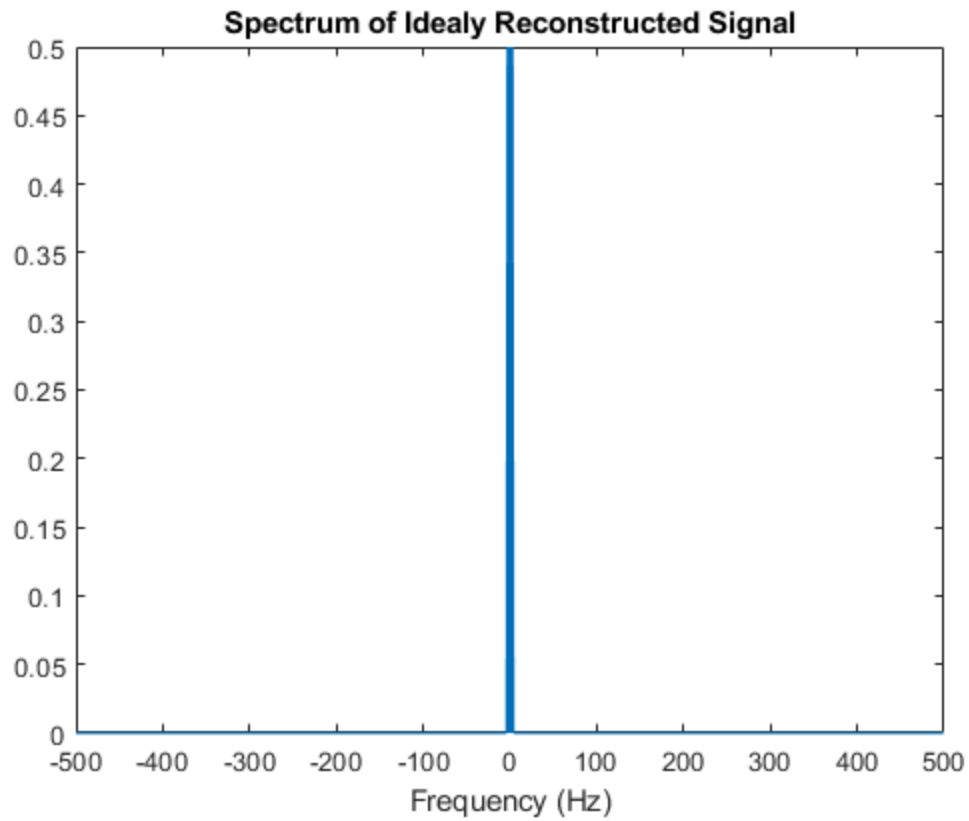
subplot(2,2,3)
hold on
plot(t,m)
plot(t,m_quantized_flat(3,:))
xlabel('Time (sec)')
title('Original Signal vs. Quantized Signal L = 16')
hold off

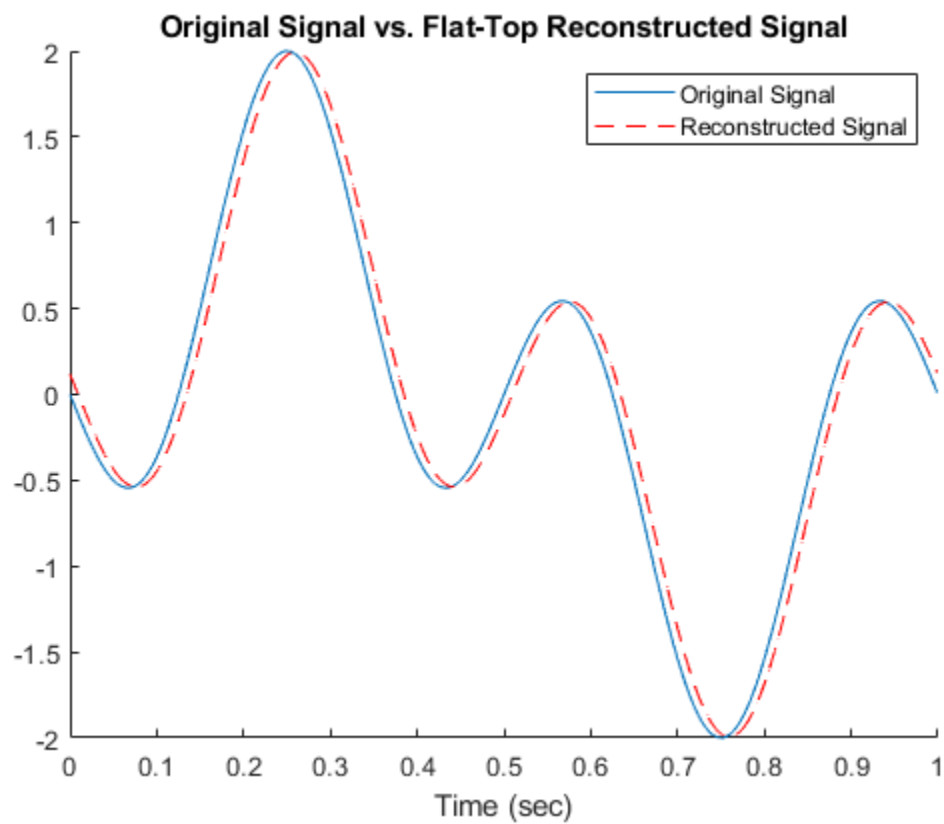
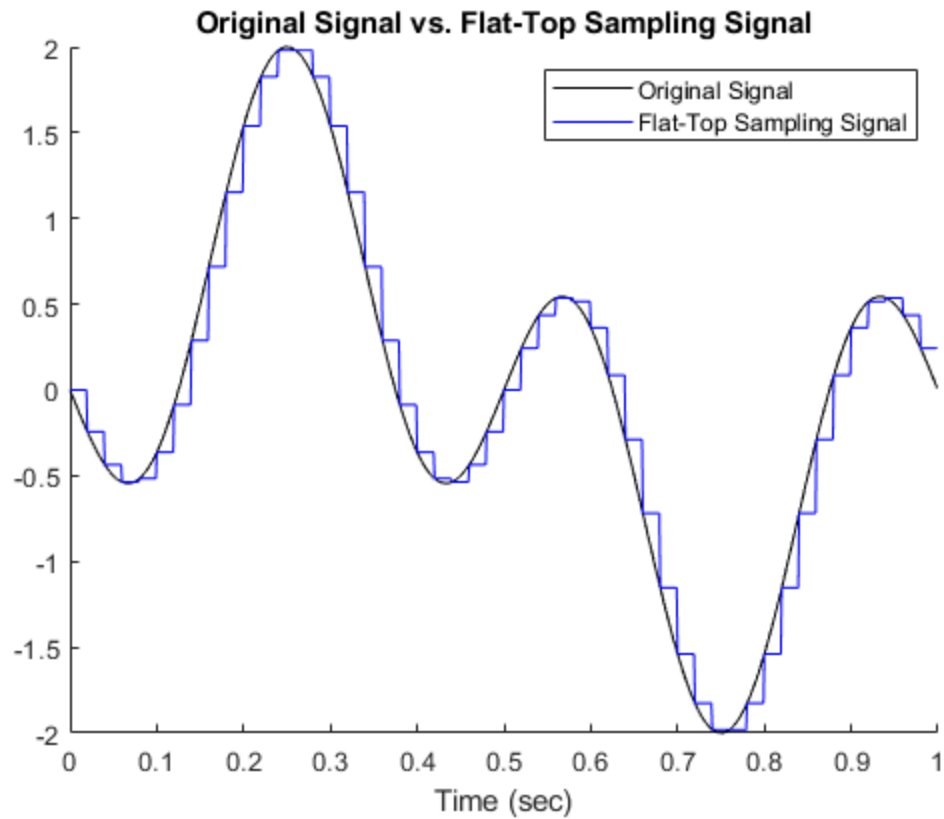
subplot(2,2,4)
hold on
plot(t,m)
plot(t,m_quantized_flat(4,:))
xlabel('Time (sec)')
title('Original Signal vs. Quantized Signal L = 32')
hold off

figure(9)
stem(k,SQNR)
xlabel('Number of bits')
ylabel('SQNR (dB)')

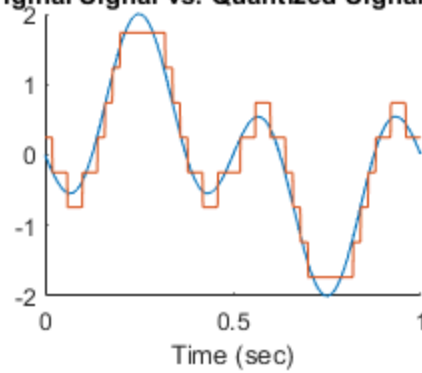
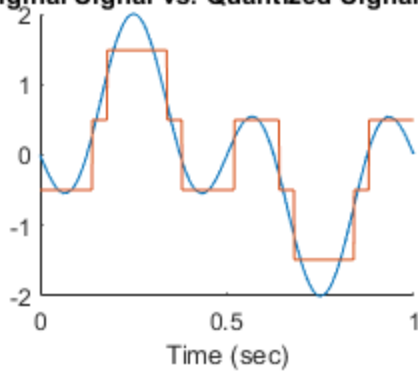
```



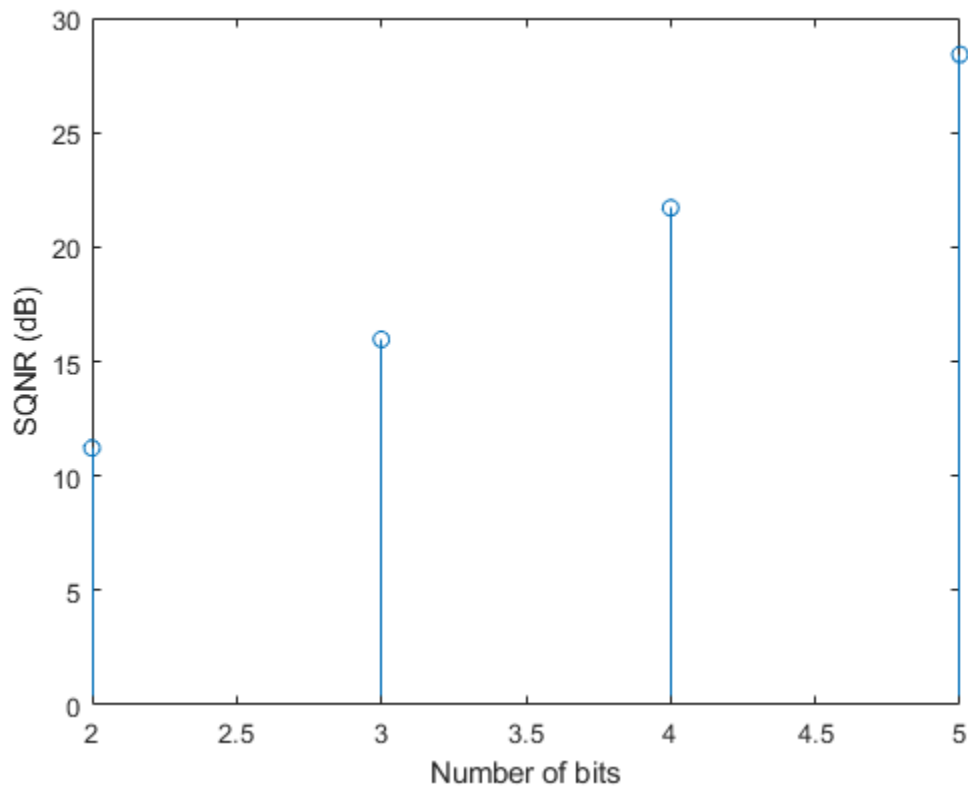
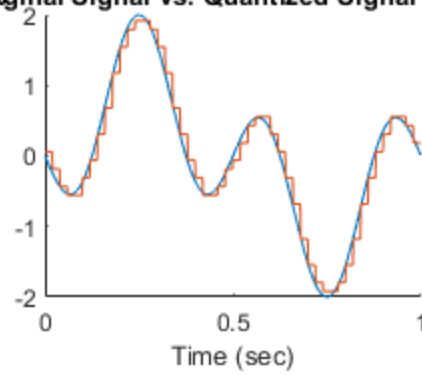
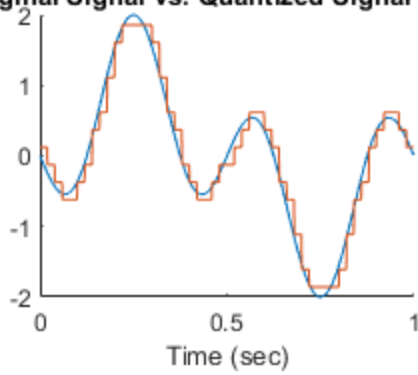




Original Signal vs. Quantized Signal L = 4 **Original Signal vs. Quantized Signal L = 8**



Original Signal vs. Quantized Signal L = 16 **Original Signal vs. Quantized Signal L = 32**



Published with MATLAB® R2022b